

Basic (a normal, exact test) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
# chunking_unit_test_practice.py
from chunking import chunk_by_paragraph

sample_text = "Paragraph one.\n\nParagraph two.\n\nParagraph three."

def test_chunk_by_paragraph():
    chunks = chunk_by_paragraph(sample_text)
    assert len(chunks) == 3
```

Run with `pytest test_chunking.py -v`. This is a correct, minimal test — it just checks the count, not the content of each chunk.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — count plus content, and a second test for the single-paragraph edge case

```
# chunking_unit_test_practice.py
from chunking import chunk_by_paragraph

sample_text = "Paragraph one.\n\nParagraph two.\n\nParagraph three."

def test_chunk_by_paragraph():
    chunks = chunk_by_paragraph(sample_text)
    assert len(chunks) == 3
    assert chunks[0] == "Paragraph one."
    assert chunks[-1] == "Paragraph three."

def test_chunk_by_paragraph_single_paragraph():
    chunks = chunk_by_paragraph("Just one paragraph, no blank lines.")
    assert len(chunks) == 1
    assert chunks[0] == "Just one paragraph, no blank lines."
```

Approach 2 — using `pytest.mark.parametrize` to cover several inputs at once

```
# chunking_unit_test_practice.py
import pytest
from chunking import chunk_by_paragraph

@pytest.mark.parametrize(
    "text,expected_count",
    [
        ("Paragraph one.\n\nParagraph two.\n\nParagraph three.", 3),
        ("Just one paragraph, no blank lines.", 1),
        ("A.\n\nB.", 2),
    ],
)
def test_chunk_by_paragraph_count(text, expected_count):
```

```
chunks = chunk_by_paragraph(text)
assert len(chunks) == expected_count
```

Difference from Basic: Approach 1 adds a second, separately-named test function for the single-paragraph edge case, rather than folding it into the same assert. Approach 2 covers the same ground and more (a third case) through one `parametrize` table, so pytest reports each row as its own pass/fail without you writing a new function per case. Neither approach yet checks anything beyond a hand-picked set of examples — that's what Advanced adds.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a fixture, a wider parametrize table, and a no-content-lost property test

```
# chunking_unit_test_practice.py
import pytest
from chunking import chunk_by_paragraph

@pytest.fixture
def sample_text() -> str:
    return "Paragraph one.\n\nParagraph two.\n\nParagraph three."

@pytest.mark.parametrize(
    "text,expected_count",
    [
        ("Paragraph one.\n\nParagraph two.\n\nParagraph three.", 3),
        ("Just one paragraph, no blank lines.", 1),
        ("", 0),
        ("A.\n\n\n\nB.", 2), # extra blank lines between paragraphs shouldn't cr
        (" \n\n ", 0), # whitespace-only input should behave like empty inp
    ],
)
def test_chunk_by_paragraph_count(text: str, expected_count: int) -> None:
    chunks = chunk_by_paragraph(text)
    assert len(chunks) == expected_count

def test_no_content_lost(sample_text: str) -> None:
    chunks = chunk_by_paragraph(sample_text)
    rejoined = " ".join(chunks)
    for word in sample_text.split():
        assert word in rejoined
```

Expected output: all 6 tests pass (`pytest test_chunking.py -v`). If `("", 0)` or `(" \n\n ", 0)` fails, that's telling you something real — some chunkers return `['']` (one empty-string chunk) instead of `[]` for empty input; decide which behavior your project actually wants and adjust the test to match it, don't just delete the case.

Approach 2 — property-based testing with Hypothesis, instead of hand-picking edge cases

```
# chunking_unit_test_practice.py
from hypothesis import given, strategies as st
from chunking import chunk_by_paragraph

@given(st.text())
def test_never_crashes_on_any_string(text: str) -> None:
    # no exception, on any string Hypothesis can generate
    chunk_by_paragraph(text)

@given(st.lists(st.text(min_size=1), min_size=1, max_size=5))
def test_no_content_lost_generated(paragraphs: list[str]) -> None:
    text = "\n\n".join(paragraphs)
    chunks = chunk_by_paragraph(text)
    rejoined = " ".join(chunks)
    for paragraph in paragraphs:
        for word in paragraph.split():
            assert word in rejoined
```

Expected output: Hypothesis runs each test 100 times (by default) against generated inputs, not just the 5 cases you'd think to write by hand. If it finds a failure, it prints the smallest input it could shrink the failure down to — often something like a single blank line or a string of just `"\n"` characters, exactly the kind of edge case easy to miss by hand.

Difference from Intermediate: Intermediate's `parametrize` table is still a list of examples a person thought of — thorough, but bounded by imagination. Approach 1 adds more hand-picked cases (empty string, whitespace-only, extra blank lines) plus a fixture and a content-preservation property, directly answering Hint 1's Advanced question about silent data loss. Approach 2 goes further: instead of a person inventing edge cases, `hypothesis` generates hundreds of random ones and actively searches for an input that breaks the property — this is a genuinely different testing strategy (property-based vs. example-based), not just "more examples."

Which one should you actually write? Approach 1's fixture-plus-parametrize-plus-property-test is what belongs in every real project's chunking test file — it's cheap, fast, and catches the specific edge cases (empty input, extra blank lines) that actually show up in real documents. Reach for Approach 2's Hypothesis only for functions where the input space is genuinely large and a subtle bug would be expensive if it shipped (a chunker feeding a production RAG pipeline is a reasonable

candidate) — it's a real dependency and a different mental model, not something to add to every small pure function by default.